

Chương 1

Giới thiệu dữ liệu lớn

Vũ Ngọc Thanh Sang

Faculty of Information Technology
Saigon University

2025

Mục tiêu bài học (Objectives)

- Hiểu được định nghĩa, nguồn gốc và đặc trưng **4V** của **Big Data**.
- Nắm bắt sự tiến hóa của Web (1.0, 2.0, 3.0) và các công nghệ web ngữ nghĩa (Semantic Web).
- Phân tích được các ứng dụng thực tế: **Search, Web, Social Network, và Mobile Analytics**.
- Hiểu các kiến trúc xử lý dữ liệu lớn: **Shared-nothing, MapReduce**.
- Nhận diện các thách thức trong điện toán đám mây: **Scalability (Định lý CAP), Fault Tolerance, và Multitenancy**.

Bối cảnh

- **Sự bùng nổ dữ liệu:**

- Năm 2007 đạt **281 exabytes**, tăng trưởng kép **57%**.
- **95%** là dữ liệu **phi cấu trúc**, đặt ra thách thức lớn trong việc lưu trữ, xử lý để phục vụ **Tìm kiếm, Phân tích, Khai phá và Trực quan hóa**.

- **Quy mô Web và Mạng xã hội:**

- **Web:** Hơn **1 nghìn tỷ** trang (**50 tỷ** được index); hơn **100 triệu** tên miền (**+20.000/ngày**).
- **Social (FB/Twitter):** Mỗi nền tảng có hơn **900 triệu** user; tạo ra hơn **300 triệu** bài đăng/ngày.

- **Giao dịch và Thiết bị số:**

- **Tài chính:** Hơn **10.000** gd/giây; **>30 tỷ** giao dịch POS/năm.
- **Di động:** Hơn **6 tỷ** điện thoại (gần **1 tỷ** smartphone kết nối Internet).
- **Đa phương tiện:** Dữ liệu hình ảnh/video (YouTube) chiếm tỷ trọng vượt xa các loại khác cộng lại.

1. Khái niệm và đặc trưng của dữ liệu lớn



1.1 Big Data là gì?

- **Định nghĩa:** Big Data là khối lượng dữ liệu có sẵn ở nhiều mức độ phức tạp khác nhau, được tạo ra với tốc độ và mức độ mơ hồ khác nhau mà không thể xử lý được bằng các công nghệ, phương pháp xử lý, thuật toán truyền thống hay bất kỳ giải pháp thương mại có sẵn nào.
- **Một cách dễ hiểu:** Big Data là tập hợp các dữ liệu rất lớn, rất nhanh và rất đa dạng, vượt quá khả năng lưu trữ, xử lý và phân tích của các hệ thống dữ liệu truyền thống.
- **VD các loại dữ liệu:**
 - Thời tiết, dữ liệu không gian địa lý và hệ thống thông tin địa lý (**GIS**).
 - Dữ liệu từ người tiêu dùng trên **mạng xã hội (social media)**.
 - Dữ liệu doanh nghiệp: pháp lý, bán hàng, marketing, tài chính, nhân sự.
 - Dữ liệu từ thiết bị: mạng cảm biến, nhà máy hạt nhân, thiết bị chụp X-quang, động cơ máy bay.

1.1.X Sự phát triển các mô hình: 3V - 4V - 5V

Mô hình 3V (Cơ bản)

Tập trung vào tính chất vật lý:

- **Volume:**
(Dung lượng lớn)
- **Velocity:**
(Tốc độ nhanh)
- **Variety:**
(Đa dạng loại)

Mô hình 4V (+Chất lượng)

Bổ sung yếu tố chất lượng do dữ liệu lớn thường bị nhiễu:

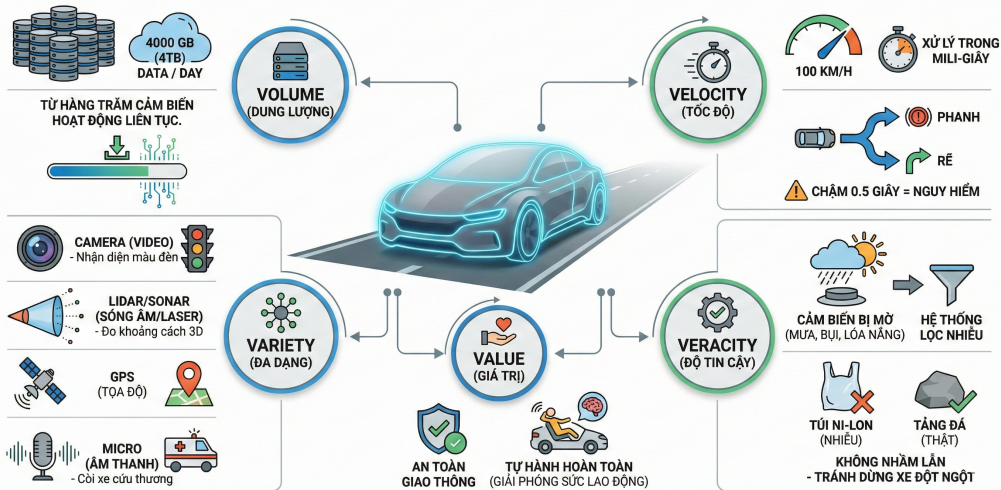
- Volume
- Velocity
- Variety
- **Veracity:**
(Tính xác thực/Độ tin cậy)

Mô hình 5V (+Mục đích)

Tiêu chuẩn hiện nay, tập trung vào hiệu quả kinh doanh:

- Volume
- Velocity
- Variety
- Veracity
- **Value:**
(Giá trị mang lại)

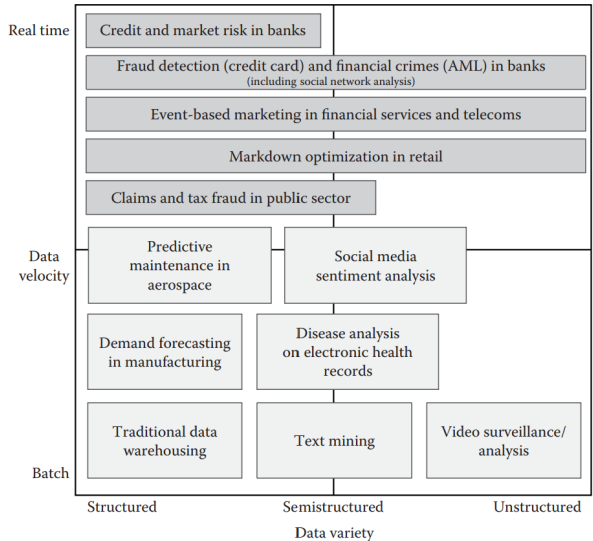
1.1.6 Ví dụ minh họa: Xe tự lái (Autonomous Car)



1.1.2 Các trường hợp sử dụng

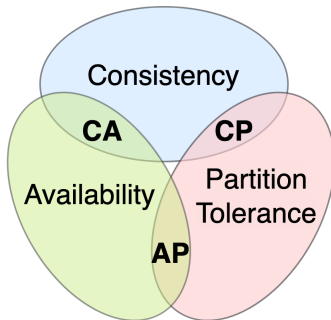
- Các trường hợp sử dụng trải dài trên nhiều lĩnh vực dựa trên tính chất dữ liệu và loại dữ liệu.
- Bao gồm: Ngân hàng (phát hiện gian lận), Bán lẻ (tối ưu hóa), Sản xuất, Y tế, và Phân tích mạng xã hội.

Figure 2: Use cases for big data computing



1.2 Định lý CAP của Brewer

Trong một hệ thống phân tán, không thể đồng thời đảm bảo cả 3 tính chất



Từ ACID đến Thách thức về Quy mô

- **Đặc tính ACID:** Là nền tảng của hệ quản trị CSDL truyền thống.
 - **Atomicity (Nguyên tử):** Giao dịch thực hiện trọn vẹn hoặc không gì cả.
 - **Consistency (Nhất quán):** Chuyển CSDL từ trạng thái hợp lệ này sang trạng thái hợp lệ khác, bảo toàn mọi ràng buộc.
 - **Isolation (Cô lập):** Giao dịch thực thi như thể nó là duy nhất trên hệ thống, không bị ảnh hưởng bởi các giao dịch đồng thời.
 - **Durability (Bền vững):** Dữ liệu đã commit sẽ tồn tại vĩnh viễn ngay cả khi có lỗi đĩa hay lỗi hệ thống.
- **Vấn đề quy mô:**
 - Khi dữ liệu và tải truy vấn quá lớn, một máy đơn lẻ không thể đáp ứng.
 - Giải pháp: Mở rộng hàng ngang bằng cách phân tán dữ liệu qua nhiều nodes thông qua phân mảnh.
- **Hệ quả:** Dẫn đến một hệ thống phân tán, nơi Eric Brewer đưa ra định lý CAP để mô tả các thuộc tính chính.

Ba thuộc tính của Định lý CAP

- **Consistency (Tính nhất quán):** Tất cả các client đều nhìn thấy cùng một dữ liệu tại cùng một thời điểm, ngay cả khi có cập nhật.
- **Availability (Tính khả dụng):** Mọi yêu cầu đều nhận được phản hồi (thành công/thất bại) từ một bản sao ngay cả khi có lỗi xảy ra.
- **Partition Tolerance (Khả năng chịu lỗi phân vùng):** Hệ thống vẫn tiếp tục hoạt động ngay cả khi mạng bị lỗi dẫn đến việc chia cắt các nút thành các nhóm riêng biệt.

Tại sao không thể có cả ba?

- **Kịch bản:** Hai máy chủ S_1, S_2 chia sẻ đối tượng o . Một thông điệp m cần truyền từ S_1 sang S_2 để cập nhật trạng thái o' .
- **Khi lỗi mạng xảy ra:** Thông điệp m không thể gửi đi.
- **Sự lựa chọn bắt buộc:**
 - **Ưu tiên Consistency:** Phải dừng việc cập nhật tại S_1 vì không thể đồng bộ sang S_2 → Mất tính **Availability**.
 - **Ưu tiên Availability:** Cho phép S_1 cập nhật nhưng S_2 sẽ mang giá trị cũ lỗi thời → Mất tính **Consistency**.
- **Kết luận:** Trong hệ thống phân tán, khi có lỗi mạng, bạn chỉ có thể chọn 2 trong 3 thuộc tính.

Phân loại các hệ thống theo CAP

- **Hệ thống CA (Consistent, Available):** Đảm bảo nhất quán và khả dụng nhưng không chịu được lỗi mạng. Thường không phù hợp cho Cloud vì lỗi mạng là tất yếu.
- **Hệ thống CP (Consistent, Partition-tolerant):** Đảm bảo nhất quán ngay cả khi phân mảnh mạng, nhưng sẽ từ chối phục vụ như ngừng hoạt động để tránh dữ liệu sai lệch.
- **Hệ thống AP (Available, Partition-tolerant):** Luôn sẵn sàng phục vụ nhưng có thể trả về kết quả không nhất quán.

Tính nhất quán cuối cùng và mô hình BASE

- **Tính nhất quán cuối cùng:** Là một lớp con của AP, hệ thống cam kết đạt trạng thái đồng bộ sau một khoảng thời gian trễ gọi là Inconsistency window nếu không có cập nhật mới.
- **Sự chuyển dịch từ ACID sang BASE:**
 - **BASE:** Viết tắt của ba đặc tính gồm Khả dụng cơ bản, Trạng thái mềm và Tính nhất quán cuối cùng.
 - Đây là giải pháp tối ưu cho Big Data khi ưu tiên tính sẵn sàng và khả năng mở rộng hơn sự nhất quán tức thời.

So sánh RDBMS và Hệ thống phân tán lớn

- **RDBMS truyền thống:** Tập trung vào ACID, ưu tiên Nhất quán hơn Khả năng chịu lỗi và Khả dụng. Sử dụng các kỹ thuật tốn kém như *Distributed Locking* hay *Two-phase commit*.
- **Thách thức Web/Cloud:** Người dùng yêu cầu hoạt động "always-on". Việc duy trì ACID trên hàng ngàn nút gây ra độ trễ cực lớn và chi phí hiệu năng không thể chấp nhận.
- **Kết luận:** Mô hình quan hệ không mở rộng tốt cho dữ liệu phi cấu trúc khổng lồ. Kỹ thuật BASE ra đời để giải quyết bài toán hiệu năng và độ trễ của Big Data.

1.3 Đặc điểm chung của các hệ thống điện toán dữ liệu lớn

1.3.1. Đặc điểm 1: Nguyên lý đồng vị trí Co-location

- **Triết lý cốt lõi:** Áp dụng tư duy di chuyển mã nguồn đến dữ liệu theo nguyên tắc Move the code to the data thay vì làm ngược lại.
- **Lý do:** Kích thước chương trình thường rất nhỏ so với các tập dữ liệu khổng lồ nên việc đọc dữ liệu tại chỗ theo phương thức locally giúp giảm thiểu lưu lượng mạng.
- **So sánh với truyền thống:**
 - **Siêu máy tính/Tính toán phân tán cũ:** Dữ liệu lưu ở kho riêng và được chuyển về hệ thống xử lý khi cần.
 - **Big Data Computing:** Dữ liệu nằm rải rác trên các nút xử lý, sau đó thuật toán được chuyển đến đúng nút chứa dữ liệu đó.
- **Lợi ích:** Giảm gánh nặng chi phí vận hành overhead và tăng hiệu suất xử lý đáng kể.

1.3.2 Đặc điểm 2: Mô hình lập trình

- **Cách tiếp cận độc lập máy chủ:**
 - Các ứng dụng được diễn đạt thông qua các thao tác dữ liệu cấp cao.
 - Hệ thống Runtime tự động quản lý việc lập lịch, thực thi, cân bằng tải và giao tiếp giữa các nút.
- **Tính trừu tượng cao:** Sử dụng các luồng dữ liệu và phép biến đổi với các thư viện thuật toán dùng chung điển hình như sorting.
- **Khác biệt với siêu máy tính truyền thống:**
 - Mô hình cũ phụ thuộc vào máy chủ, yêu cầu lập trình viên kiểm soát cấp thấp việc giao tiếp giữa các nút gây phức tạp và giảm năng suất.
 - Mô hình cũ dễ gặp lỗi điểm đơn single point of failure và cần tinh chỉnh thủ công rất nhiều.

1.3.3 Đặc điểm 3: Tập trung vào Độ tin cậy và Tính khả dụng

- **Thực trạng:** Các hệ thống quy mô lớn với hàng ngàn nút rất dễ gặp lỗi phần cứng, lỗi mạng hoặc lỗi phần mềm.
- **Khả năng phục hồi:**
 - Duy trì các bản sao dự phòng của mọi tệp dữ liệu trên đĩa.
 - Lưu trữ các kết quả xử lý trung gian để tránh làm lại từ đầu.
 - Tự động phát hiện lỗi nút và thực hiện tính toán lại có chọn lọc.
- **Duy trì vận hành:** Cụm xử lý có thể tiếp tục hoạt động với số lượng nút giảm đi sau sự cố nhờ khả năng phục hồi tự động và trong suốt.

1.3.4. Khả năng mở rộng và Vai trò của Cloud

- **Mở rộng tuyến tính Linear Scalability:**

- Có thể mở rộng để đáp ứng hầu như mọi lượng dữ liệu bằng cách thêm các nút xử lý mới.
- Đạt tốc độ xử lý hàng tỷ bản ghi mỗi giây tương đương chỉ số BORPS.

- **Sự kết hợp với Cloud:**

- **Tính đàn hồi Elasticity:** Người dùng có thể thêm hoặc bớt tài nguyên theo thời gian thực dựa trên nhu cầu.
- **Công nghệ Cloudware:** Kỹ thuật ảo hóa Virtualization giúp tăng hiệu suất và tối ưu hóa các hệ thống phức tạp.
- **Tác động:** Biến các bài toán vốn bị coi là bất khả thi vì khối lượng dữ liệu quá lớn thành có thể thực hiện được, tạo bước đột phá trong phân tích thông tin.

2. Ứng dụng của dữ liệu lớn trong thực tế



2.1: Web và Ứng dụng Doanh nghiệp - Nơi khởi nguồn của Dữ liệu lớn

Nền tảng J2EE và Khả năng mở rộng

Các hệ thống lớn thường dựa trên J2EE (Java 2 Platform, Enterprise Edition)
- nền tảng chuẩn hóa để xây dựng ứng dụng doanh nghiệp phân tán.

- **Kiến trúc đa tầng (N-tier):** Tách biệt giao diện, xử lý và dữ liệu giúp hệ thống phục vụ hàng triệu người dùng (Client) cùng lúc.
- **Hệ quả dữ liệu:** Khả năng mở rộng này dẫn đến bùng nổ số lượng giao dịch (Transaction) mỗi giây.
- **Thách thức:** Các thành phần như Enterprise Java Beans phải đảm bảo tính nhất quán của dữ liệu giao dịch, tạo ra áp lực lưu trữ khổng lồ cho cơ sở dữ liệu.

Mô hình MVC và Phân tách dữ liệu

MVC (Model-View-Controller) là kiến trúc phần mềm chia ứng dụng thành ba phần để tách biệt mối quan tâm.

- **Model (Dữ liệu):** Chứa logic nghiệp vụ và dữ liệu gốc.
- **View (Giao diện):** Hiển thị dữ liệu cho người dùng.
- **Controller (Điều khiển):** Xử lý tương tác và cập nhật Model.
- **Ý nghĩa Big Data:** Việc tách biệt Model giúp thu thập dữ liệu thô dễ dàng hơn để đưa vào Kho dữ liệu (Data Warehouse) mà không bị lẫn mã giao diện HTML.

Tiến hóa Web: Từ Web 1.0 đến Web 2.0

Sự chuyển dịch hành vi người dùng là động lực chính của Big Data.

- **Web 1.0 (Read-only):** Người dùng chỉ đọc nội dung tĩnh. Dữ liệu chủ yếu là văn bản và log truy cập.
- **Web 2.0 (Read-Write):** Kỷ nguyên tương tác. Người dùng tự tạo nội dung (User Generated Content - UGC) như bài viết, bình luận, video.
- **Hệ quả:** Sự xuất hiện của dữ liệu phi cấu trúc chiếm dụng băng thông và dung lượng lưu trữ lớn.

Web 3.0 và Semantic Web

Web 3.0 (Semantic Web) hướng tới việc máy tính có thể "hiểu" dữ liệu.

- **Semantic Markup (Đánh dấu ngữ nghĩa):** Gắn thẻ metadata để máy tính phân biệt được con số "1000" là giá tiền hay khoảng cách.
- **Ontology (Bản thể học):** Bộ quy tắc định nghĩa mối quan hệ giữa các khái niệm, giúp máy móc tự động suy luận (ví dụ: A là bố B, B là bố C → A là ông nội C).
- **Giá trị:** Biến Internet thành một cơ sở dữ liệu tri thức khổng lồ có thể liên kết tự động.

Mối liên hệ với Big Data

Công nghệ Web đóng góp hai chữ V quan trọng đầu tiên:

- **VOLUME:**

- J2EE/MVC cho phép phục vụ hàng triệu người dùng → Hàng tỷ bản ghi giao dịch (Transactional Data).
- Web 2.0 khuyến khích upload media → Kho lưu trữ phình to (Multimedia Data).

- **VARIETY:**

- Chuyển từ dữ liệu cấu trúc (SQL) sang dữ liệu bán cấu trúc (XML, JSON của Web 3.0) và phi cấu trúc (Video, ảnh của Web 2.0).

2.2: Phân tích Web - Theo dấu chân người dùng số

Nhật ký Tìm kiếm (Search Log)

Nhật ký máy chủ (Log files) ghi lại mọi dấu vết kỹ thuật số.

- **Thành phần:** IP người dùng, thời gian (Timestamp), từ khóa truy vấn (Query) và trang đích (Landing page).
- **Đặc điểm:** Dữ liệu được sinh ra liên tục, tự động và có cấu trúc thời gian.
- **Quy mô:** Với các công cụ tìm kiếm lớn (Google, Bing), lượng log này đạt hàng Petabyte mỗi ngày.

Quy trình SLA (Search Log Analysis)

SLA là quy trình khai thác tri thức từ nhật ký tìm kiếm.

- **Bước 1 - Thu thập:** Ghi nhận tương tác ngầm (Passive logging) để không ảnh hưởng trải nghiệm.
- **Bước 2 - Tiền xử lý:** Loại bỏ truy vấn từ Robot/Bot, gộp các hành động rời rạc thành một phiên làm việc.
- **Bước 3 - Phân tích:** Sử dụng thống kê để hiểu ý định người dùng (User Intent) qua độ phức tạp của từ khóa và tần suất tìm kiếm lại.

Các chỉ số đo lường (Web Metrics)

Đo lường hiệu quả website đòi hỏi tính toán trên dữ liệu lớn:

- **Conversion Rate (Tỷ lệ chuyển đổi):** % người dùng thực hiện hành động mục tiêu (mua hàng).
- **Bounce Rate (Tỷ lệ thoát):** % người dùng rời đi ngay sau khi xem một trang (thể hiện sự không hài lòng).
- **Thách thức định danh:** Việc theo dõi một người dùng chuyển đổi qua lại giữa điện thoại và laptop đòi hỏi kỹ thuật ghép nối dữ liệu phức tạp (Cross-device tracking).

Mối liên hệ với Big Data

Phân tích Web là minh chứng rõ nhất cho tốc độ xử lý:

- **VELOCITY:**

- Dữ liệu Log sinh ra liên tục 24/7 theo thời gian thực.
- Hệ thống phải xử lý (capture & store) ngay lập tức, chậm trễ sẽ dẫn đến mất mát dữ liệu hoặc bỏ lỡ cơ hội quảng cáo.

- **Data Granularity (Độ mịn dữ liệu):**

- Thay vì chỉ lưu kết quả cuối cùng (đơn hàng), Big Data lưu lại toàn bộ hành trình (click, scroll, hover) dẫn đến đơn hàng đó.

2.3: Mạng xã hội - Cấu trúc đồ thị và Giá trị ẩn

Nguyên lý vận hành của Mạng lưới

Dữ liệu mạng xã hội tuân theo các quy luật toán học đặc thù:

- **Định luật Metcalfe:** Giá trị của mạng tăng theo bình phương số lượng người dùng (n^2).
- **Power Law (Luật lũy thừa):** Một số ít người nổi tiếng (Hubs) tạo ra phần lớn tương tác, trong khi đa số người dùng ít hoạt động (Long Tail).
- **Small World (Thế giới nhỏ):** Khoảng cách kết nối giữa hai người bất kỳ rất ngắn, tạo điều kiện cho tin tức lan truyền cực nhanh.

Phân tích Mạng xã hội (SNA)

SNA (Social Network Analysis) là kỹ thuật nghiên cứu cấu trúc xã hội qua lý thuyết đồ thị.

- **Dữ liệu đồ thị (Graph Data):** Biểu diễn bằng các Nút (Node - con người) và Cạnh (Edge - mối quan hệ) thay vì bảng dòng/cột truyền thống.
- **Chỉ số trung tâm (Centrality):** Xác định ai là người ảnh hưởng nhất (Influencer) hoặc ai là cầu nối (Bridge) giữa các nhóm cộng đồng.
- **Ứng dụng:** Phát hiện gian lận (các nhóm tội phạm), gợi ý kết bạn, marketing lan truyền.

Lý thuyết Liên kết yếu (Weak Ties)

Giá trị của dữ liệu nằm ở các mối quan hệ xã giao:

- **Strong Ties (Liên kết mạnh):** Bạn thân/gia đình thường chia sẻ thông tin giống nhau → Ít thông tin mới.
- **Weak Ties (Liên kết yếu):** Người quen xã giao là cầu nối mang lại thông tin mới, cơ hội việc làm từ các nhóm xã hội khác.
- **Khai thác:** Thuật toán Big Data tận dụng liên kết yếu để mở rộng phạm vi gợi ý nội dung thay vì chỉ xoay quanh những gì người dùng đã biết.

Mối liên hệ với Big Data

Mạng xã hội nâng tầm độ phức tạp và giá trị dữ liệu:

- **VARIETY:**

- Sự chuyển dịch từ cơ sở dữ liệu quan hệ (RDBMS) sang cơ sở dữ liệu đồ thị (Graph DB) để lưu trữ mối quan hệ phức tạp.

- **VALUE:**

- Dữ liệu rời rạc của một cá nhân ít giá trị, nhưng khi đặt trong mối quan hệ mạng lưới (SNA), nó tiết lộ xu hướng cộng đồng và sức ảnh hưởng - chìa khóa của Marketing hiện đại.

2.4: Di động - Thách thức về Độ tin cậy và Thời gian thực

Phân tích Di động (Mobile Analytics)

Thiết bị di động là nguồn phát dữ liệu cá nhân hóa và đa chiều nhất.

- **Đa nguồn:** Kết hợp dữ liệu cảm biến phần cứng (GPS, gia tốc kế), nhật ký cuộc gọi và tương tác ứng dụng.
- **Thách thức môi trường:** Dữ liệu thường bị ngắt quãng do mất sóng, hết pin hoặc người dùng di chuyển nhanh.
- **Mục tiêu:** Tối ưu hóa trải nghiệm trên màn hình nhỏ và cung cấp dịch vụ đúng ngữ cảnh (Context).

Khai phá Văn bản (Text Mining) và NLP

Tin nhắn và bình luận trên di động tạo ra thách thức lớn về xử lý.

- **NLP (Natural Language Processing - Xử lý ngôn ngữ tự nhiên):** Kỹ thuật giúp máy tính đọc hiểu ngôn ngữ con người.
- **Đặc thù di động:** Văn bản chứa nhiều tiếng lóng, viết tắt, sai chính tả và biểu tượng cảm xúc (Emoji).
- **Sentiment Analysis (Phân tích cảm xúc):** Đánh giá thái độ người dùng (tích cực/tiêu cực) từ văn bản phi cấu trúc để hiểu phản hồi khách hàng.

Phân tích Luồng (Streaming Analysis)

Khác với xử lý theo lô (Batch Processing), Streaming xử lý dữ liệu đang chuyển động.

- **Deductive (Suy diễn):** Áp dụng quy tắc có sẵn (ví dụ: Nếu quẹt thẻ ở 2 nước khác nhau trong 1 giờ → Chặn thẻ).
- **Inductive (Quy nạp):** Vừa xử lý vừa học từ dữ liệu để cập nhật mô hình dự báo.
- **Yêu cầu:** Độ trễ (Latency) phải cực thấp, tính bằng mili-giây.

Mối liên hệ với Big Data

Di động đặt ra thách thức lớn về chất lượng dữ liệu:

- **VERACITY:**

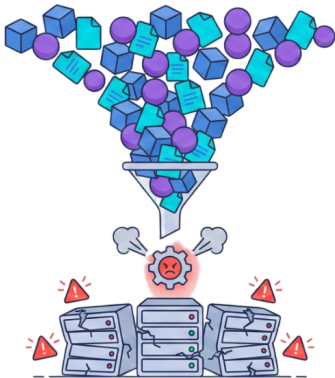
- Dữ liệu văn bản di động rất "nhiều" (Noisy data).
- Cảm biến có thể sai số.
- → Big Data cần các thuật toán làm sạch và NLP mạnh mẽ để lọc bỏ nhiễu trước khi phân tích.

- **VELOCITY:**

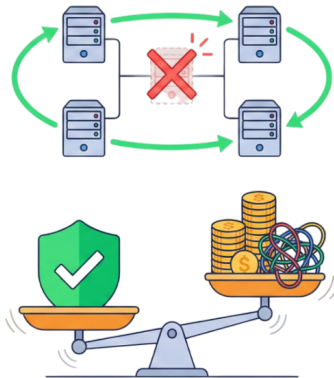
- Streaming Analysis là bắt buộc. Giá trị thông tin sẽ bằng 0 nếu xử lý chậm (ví dụ: cảnh báo kẹt xe sau khi người dùng đã đi vào đường kẹt).

3. Thách thức trong việc xử lý dữ liệu lớn

The Volume Challenge



Reliability & Fault Tolerance

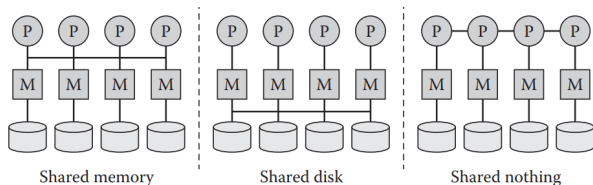


3.1 Phương pháp Xử lý

- Các nền tảng Big Data hiện tại sử dụng phương pháp xử lý song song **"chia để trị"**.
- Kết hợp nhiều bộ xử lý và đĩa trong các cụm tính toán lớn (clusters) kết nối qua mạng tốc độ cao.
- **Định nghĩa Cluster:** Một tập hợp các máy tính độc lập được kết nối với nhau, làm việc như một tài nguyên tính toán tích hợp duy nhất.
- **Cách tiếp cận "Shared nothing":**
 - Mỗi node (gồm bộ xử lý, bộ nhớ, đĩa) không chia sẻ tài nguyên với các node khác.
 - Phù hợp cho các bài toán **"embarrassingly parallel"** (dễ dàng tách thành các tác vụ song song độc lập mà không cần giao tiếp nhiều).

Phân loại Kiến trúc Song song

- Kiến trúc song song được phân loại dựa trên tài nguyên chia sẻ mà mỗi bộ xử lý có thể truy cập trực tiếp.
- 3 loại kiến trúc chính:
 1. **Shared memory**
 2. **Shared disk**
 3. **Shared nothing**



Parallel architectures

Shared Memory & Shared Disk

- **Shared Memory System:**

- Tất cả bộ xử lý truy cập bộ nhớ qua một **bus chia sẻ**.
- Ví dụ: Hệ thống đa xử lý đối xứng (**SMP**).
- **Hạn chế:** Phải duy trì tính nhất quán của bộ nhớ đệm (cache consistency) và giới hạn băng thông bus.

- **Shared Disk Architecture:**

- Mỗi bộ xử lý có bộ nhớ riêng (private memory), nhưng chia sẻ tất cả các đĩa.
- Ví dụ: Cụm máy tính kết nối với **SAN** (Storage Area Network).
- Vẫn gặp hiện tượng "nghẽn cổ chai" (bottlenecks) do chia sẻ đường truyền I/O.

Kiến trúc Shared Nothing

- Mỗi bộ xử lý có bộ nhớ riêng và đĩa riêng. Dữ liệu được phân tán trên tất cả các đĩa.
- Nếu một bộ xử lý bị lỗi, dữ liệu trên node đó sẽ không khả dụng (cần cơ chế dự phòng).
- **Ưu điểm vượt trội:**
 - Là kiến trúc **có khả năng mở rộng nhất (most scalable)** vì không có môi trường chia sẻ trung gian.
 - Là lựa chọn số một cho các truy vấn phân tích (analytical queries).
 - Phù hợp với kiến trúc **MPP** (Massively Parallel Processing), tránh được tắc nghẽn băng thông bus mà SMP và Shared-disk gặp phải.

3.2 Chiến lược lưu trữ và xử lý

3.2.1 Đặc điểm các phương pháp lưu trữ big data (1/2)

- **1. Dữ liệu được phân tán trên nhiều node (Data distributed):**

- Big Data sử dụng các tập dữ liệu lớn hơn nhiều, phân tán trên các máy tính phổ thông (commodity machines) với dung lượng 2–4 Tb.
- **Lợi ích:**
 - **Khả năng chịu lỗi:** Mỗi khối dữ liệu được nhân bản (replication) trên nhiều node (Hadoop mặc định là 3).
 - **Hiệu suất:** Xử lý song song trên nhiều node giúp cải thiện hiệu suất 5–10 lần.

- **2. Ứng dụng di chuyển đến node chứa dữ liệu:**

- Khác với kiến trúc 3 tầng truyền thống (chuyển dữ liệu đến tầng ứng dụng), Big Data chuyển ứng dụng (và thư viện) đến nơi chứa dữ liệu.
- **Lý do:** Tránh bão hòa mạng khi di chuyển hàng Terabytes dữ liệu, ngăn ngừa lỗi hệ thống.

3.2.1 Đặc điểm các phương pháp lưu trữ big data (2/2)

- **3. Xử lý dữ liệu cục bộ (Data processed local):**

- Dữ liệu được xử lý tại chỗ hoặc càng gần dữ liệu càng tốt.
- Kết quả sau xử lý thường nhỏ hơn nhiều so với dữ liệu thô, giúp chi phí mạng ở mức quản lý được.

- **4. Ưu tiên đọc tuần tự (Sequential reads):**

- Khác với RDBMS (hướng truy cập ngẫu nhiên), hệ thống Big Data ưu tiên truy cập tuần tự để tối ưu hóa thông lượng (throughput) cho toàn hệ thống.

3.2.2 Phương pháp xử lý

1. **Massively Parallel Processing (MPP):**

- Sử dụng cách phân chia dữ liệu định trước (theo cột hoặc giá trị).
- **Hạn chế:** Khó khăn cho các truy vấn bất chợt.
- **Giải pháp:** Lưu trữ dữ liệu nhiều lần với các tiêu chí phân chia khác nhau.

2. **In-memory database systems:**

- Giống MPP in-memory với giao diện SQL (Ví dụ: SAP HANA).
- Node có bộ nhớ lớn, dữ liệu được tải sẵn vào RAM.
- **Nhược điểm:** Khóa phần cứng/phần mềm và chi phí cao.

3. **Bulk Synchronous Parallel (BSP):**

- Các tiến trình đồng bộ hóa tại một rào chắn, gửi dữ liệu cho Master node.
- Master node điều phối việc tiếp tục hay hủy bỏ dựa Checkpoint.

4. **MapReduce systems:**

- Người dùng định nghĩa quy trình Map và Reduce.
- Dữ liệu chia thành các khối (64–128 Mb), nhân bản 2 lần (hệ số 3).
- Thư viện ứng dụng được chuyển đến từng node để xử lý.

Ví dụ quy trình MapReduce (Sales Data)

- **Bài toán:** Tính doanh số năm 2014 theo quốc gia.
- **Bước 1 (Map):**
 - Mapper quét các khối dữ liệu, lọc bản ghi năm 2014.
 - **Output:** Key-value pair (Key: Country, Value: Sales number).
- **Bước 2 (Reduce):**
 - Các Key giống nhau được gửi đến cùng một Reducer.
 - Reducer cộng tổng giá trị sales cho mỗi key.
 - **Output:** Ghi lại vào HDFS (Key: Country, List of values).
- **Lưu ý:** Client sắp xếp kết quả sau cùng. Giao tiếp giữa Mapper và Reducer gây ra Network I/O.

3.3 Khả năng chịu lỗi

- **Định nghĩa:** Là khả năng của một hệ thống tiếp tục thực hiện các chức năng dự kiến khi có lỗi xảy ra.
- Liên quan đến độ tin cậy (reliability), hoạt động thành công và sự vắng mặt của sự cố hỏng hóc.
- **Vấn đề cơ bản:** Khi độ phức tạp tăng, độ tin cậy giảm mạnh nếu không có biện pháp bù đắp.
- **Ví dụ tính toán độ tin cậy:**
 - Linh kiện đơn lẻ: 99.99%.
 - Hệ thống 100 linh kiện (không dư thừa): $0.9999^{100} \approx 99.01\%$.
 - Hệ thống 10.000 linh kiện (không dư thừa): $0.9999^{10000} \approx 36.79\%$.
- Mức độ tin cậy thấp này là không thể chấp nhận được trong hầu hết các ứng dụng.

3.3.1 Lý do hệ thống vẫn thất bại

- **Thực tế:** Không thể xây dựng một hệ thống hoàn hảo tuyệt đối.
- **Nguyên nhân lỗi:**
 - Yếu tố môi trường không mong đợi hoặc chưa được tính đến.
 - Sai lầm tiềm ẩn của người dùng không được lường trước.
 - Ngay cả khi thiết kế hoàn hảo, các lỗi vẫn có thể xảy ra do các tình huống nằm ngoài tầm kiểm soát của nhà thiết kế.
- **Định nghĩa thất bại (Failure):** Khi hệ thống ngừng thực hiện chức năng dự kiến hoặc thực hiện với chất lượng/số lượng dưới mức bình thường (suy giảm, mất ổn định).
- **Mục tiêu:** Giảm thiểu xác suất thất bại, dù là gây phiền toái hay thảm họa.

3.3.2 Phân loại ứng dụng yêu cầu Khả năng chịu lỗi

Dù phần cứng ngày càng tin cậy, Khả năng chịu lỗi vẫn thiết yếu cho các loại hệ thống sau:

- **Safety-critical (An toàn là trên hết):** Tránh thiệt hại nhân mạng hoặc môi trường.
 - Ví dụ: Nhà máy điện hạt nhân, máy xạ trị, máy tạo nhịp tim, kiểm soát bay.
- **Mission-critical (Nhiệm vụ là trên hết):** Nhấn mạnh việc hoàn thành sứ mệnh (Ví dụ: Tàu vũ trụ, vệ tinh).
- **Business-critical (Kinh doanh là trên hết):** Đảm bảo hoạt động kinh doanh liên tục (Ví dụ: Ngân hàng, chứng khoán, E-commerce).

3.3.3 Thách thức Kỹ thuật

Sự quan tâm đến Khả năng chịu lỗi hồi sinh mạnh mẽ từ giữa những năm 1990 - nay do:

- **Phần cứng (Hardware):**

- Quy trình sản xuất nhỏ lại, điện áp thấp, tốc độ cao → giảm biên độ nhiễu (noise margins).
- IC trở nên nhạy cảm với **lỗi thoáng qua (transient faults)** do nhiễu xuyên âm (crosstalk) và bức xạ môi trường (neutron, hạt alpha).

- **Phần mềm (Software):**

- Sự phát triển của tính toán thời gian thực và thiết bị nhúng thông minh.
- Phần mềm linh hoạt, không bị "mòn" (wear out) như phần cứng nhưng dễ gặp lỗi thiết kế.

3.3.4 Sự dư thừa (Redundancy)

- Điểm chung của mọi phương pháp chịu lỗi là **Sự dư thừa**.
- **Định nghĩa:** Cung cấp khả năng chức năng không cần thiết trong môi trường không có lỗi (ví dụ: linh kiện sao chép, bit kiểm tra, mã xác minh).
- **Hai loại dư thừa chính:**
 - **Space redundancy (Không gian):** Thêm phần cứng, phần mềm, hoặc thông tin (Hardware/Software/Information redundancy).
 - **Time redundancy (Thời gian):** Lặp lại tính toán hoặc truyền dữ liệu và so sánh kết quả.

3.4 Nghịch lý về độ tin cậy (Reliability Conundrum)

Khái niệm về độ tin cậy

- **Định nghĩa chung:** Là khả năng hệ thống thực hiện các chức năng yêu cầu trong những điều kiện nhất định và một khoảng thời gian xác định.
- **Trong phần mềm:** Là khả năng ứng dụng và toàn bộ hạ tầng phụ thuộc (HĐH, server, mạng, nguồn điện...) thực thi không lỗi cho đến khi hoàn tất.
- **Nghịch lý ở quy mô lớn:**
 - Ngay cả khi phần mềm hoàn hảo, việc sử dụng hàng ngàn máy chủ sẽ dẫn đến xác suất lỗi phần cứng tăng cao do chạm ngưỡng **MTTF (Thời gian trung bình đến khi hỏng)**.
 - Hệ quả: Ứng dụng phụ thuộc vào các máy chủ này cũng sẽ thất bại theo.

Kỹ thuật thiết kế dựa trên dự phòng (Redundancy)

- **Dự phòng phần mềm (Software Redundancy):**
 - Sử dụng các thành phần dự phòng gấp đôi hoặc gấp ba chạy song song với các bước kiểm tra xác thực chung.
 - **Chiến lược:** Các nhóm khác nhau phát triển cùng một đặc tả để tránh lỗi logic chung.
 - **Quy tắc đa số (Majority rule):** Nhiều tiến trình song song tự kiểm tra lẫn nhau trong môi trường vận hành.
- **Dự phòng phần cứng (Hardware Redundancy):**
 - Sử dụng **RAID** cho lưu trữ, dự phòng giao diện mạng và nguồn điện.
 - Ưu tiên thiết bị phần cứng tiêu chuẩn (commodity hardware) để dễ dàng lắp đặt và thay thế.

Các dịch vụ hỗ trợ độ tin cậy

- **Clustering (Phân cụm):** Liên kết nhiều máy tính để hoạt động như một thực thể đơn lẻ có tốc độ cao hơn.
- **Load Balancing (Cân bằng tải):** Giữ khối lượng công việc được phân bổ đều giữa các máy tính.
- **Data Replication (Sao chép dữ liệu):** Tạo nhiều bản sao dữ liệu giống hệt nhau để xử lý độc lập và song song.
- **Transactions (Giao dịch):** Bảo vệ các thao tác phức tạp để đảm bảo tính toàn vẹn của quy trình.
- → Các dịch vụ này thường được tích hợp sẵn trong hạ tầng của các nhà cung cấp Cloud.

Trường hợp thực tế: Bài toán của Google (2008)

- **Thách thức:** Tìm kiếm trên quy mô 1 nghìn tỷ URL duy nhất.
- **Giải pháp phần cứng:** Sử dụng các cụm máy tính giá rẻ (commodity) hoạt động song song trên tập dữ liệu lớn.
- **Vấn đề:** Với hàng trăm nghìn máy chủ, lỗi xảy ra hàng ngày là không thể tránh khỏi do đạt ngưỡng MTTF.
- **Thay đổi tư duy:** Google từ bỏ việc tìm kiếm độ tin cậy ở phần cứng và chuyển sang xây dựng độ tin cậy ở **mức phần mềm**.
- → Một hệ thống tin cậy có thể được xây dựng trên một nhóm các máy tính không tin cậy.

MapReduce và Lập trình song song

- **MapReduce:** Framework phần mềm được Google phát minh để giải quyết vấn đề lỗi phần cứng trong các cụm máy tính lớn.
- **Nguồn gốc:** Lấy cảm hứng từ hàm *map* và *reduce* của ngôn ngữ lập trình Lisp.
- **Lợi ích kép:**
 - Giải quyết vấn đề độ tin cậy thông qua cơ chế xử lý lỗi của phần mềm.
 - Mang lại sự gia tăng đột phá về hiệu suất nhờ tính song song hóa khổng lồ.
- → Nếu không có tính song song, việc xử lý các tập dữ liệu web khổng lồ là không thể thực hiện được.

Minh họa từ giao thức TCP/IP

- Chiến lược "chấp nhận lỗi thay vì tránh lỗi" đã tồn tại từ lâu trong kiến trúc Internet:
- **IP (Internet Protocol):** Di chuyển các gói dữ liệu giữa các điểm một cách **không tin cậy** (có thể thất lạc gói tin).
- **TCP (Transmission Control Protocol):** Xây dựng kết nối **tin cậy** bên trên IP bằng cách chấp nhận việc thất lạc và gửi lại các gói tin khi cần thiết.
- → Đây là minh chứng cho việc dùng phần mềm/giao thức để tạo ra sự tin cậy từ hạ tầng không tin cậy.

3.5 Dịch vụ quản lý

Dịch vụ Quản lý

- Dịch vụ quản lý bao gồm các thành phần chính:
 - Triển khai (Deployment).
 - Giám sát (Monitoring).
 - Báo cáo (Reporting).
 - Cam kết mức dịch vụ (Service-level agreement - SLA).
 - Đo lường và tính cước (Metering billing).

3.5.1 Triển khai và Cấu hình

- Cần quy trình tự động hóa vòng đời để giảm sự phức tạp và gánh nặng quản trị.
- **RightScale Cloud Management Platform:** Giải quyết 3 giai đoạn: Thiết kế, Quản lý và Triển khai.
- **Tự động hóa cấu hình:**
 - Cải thiện tính bền vững (robustness) và chức năng mà không hy sinh khả năng tự sửa chữa.
 - Công cụ quản lý giúp lập trình viên quản lý cấu hình máy chủ/ứng dụng bằng **mã (code)** thay vì thao tác thủ công.

3.5.2 Giám sát và Báo cáo

- Việc phát triển, kiểm thử và nghiên cứu hiệu năng hệ thống đám mây rất phức tạp; chi phí quản lý tăng theo số lượng site.
- **Cơ chế giám sát:**
 - Theo dõi vòng đời SLA.
 - Xác định khi SLA hoàn thành và báo cáo cho dịch vụ tính cước.
- **Ví dụ: Hyperic HQ** giám sát các dịch vụ Amazon (SimpleDB, SQS, FPS), thu thập ma trận dữ liệu và cung cấp phân tích báo cáo phong phú.

3.5.3 Quản lý SLA

- Người dùng mong muốn dịch vụ ổn định (24/7), nhưng hầu hết nhà cung cấp không đảm bảo tính sẵn sàng cao tuyệt đối.
- **Cơ chế bồi thường:** Nhà cung cấp phải trả phạt/bồi thường nếu dịch vụ bị gián đoạn.
- **SLA quy định tiêu chuẩn về:**
 1. Cách sử dụng thước đo chất lượng thô để đánh giá thành phần dịch vụ.
 2. Cách xác định thước đo chất lượng thô thành thước đo chất lượng dịch vụ.
 3. Cách ước tính mức độ chất lượng dịch vụ.
 4. Cách báo cáo kết quả đánh giá.
 5. Cách giải quyết tranh chấp.
- **Thực tế:** Amazon S3 (99.9% uptime, tín dụng giới hạn 10-25%), 3Tera VPDC (99.999%).

3.5.4 Đo lường và Tính cước

- Tính cước minh bạch giúp tăng niềm tin của người dùng.
- **Mô hình phổ biến:** Pay-as-you-go (Trả theo mức dùng) hoặc Pay-as-you-consume.
- Quy trình: Nhận trạng thái từ SLA → Gọi dịch vụ tin dụng → Trừ tiền tài khoản → Thông báo người dùng.
- **Chiến lược giá:** Giờ RAM, dung lượng CPU, băng thông, không gian lưu trữ, phí bản quyền phần mềm.

3.6 Thách thức về điện toán đám mây

3.6 Thách thức của Đám mây

- Bất kỳ nền tảng đám mây nào cũng cần giải quyết các thách thức kỹ thuật để cung cấp hạ tầng như một tiện ích thực thụ.
- **Ba thách thức chính:**
 - **Scalability (Khả năng mở rộng):** Khả năng mở rộng để đáp ứng hàng triệu khách hàng truy cập đồng thời.
 - **Multitenancy (Đa thuê bao):** Khả năng cung cấp sự cô lập và hiệu suất tốt cho nhiều khách thuê sử dụng chung hạ tầng.
 - **Availability (Tính sẵn sàng):** Đảm bảo hạ tầng và ứng dụng luôn sẵn sàng bất chấp lỗi phần cứng hay phần mềm.

3.6.1 Khả năng mở rộng

Việc mở rộng tính toán có thể thực hiện ở hai cấp độ:

- **Cấp độ hạ tầng (Infrastructure level):** Tăng năng suất của sức mạnh tính toán thô.
- **Cấp độ nền tảng (Platform level):** Quản lý thông minh các yêu cầu của khách hàng để tối ưu hóa hạ tầng mà không cần khách hàng can thiệp.

Hai phương thức mở rộng chính:

- **Mở rộng hàng dọc (Scale-up/Vertical):** Thêm tài nguyên (CPU, RAM) vào một nút duy nhất. Ưu điểm: Mô hình lập trình đơn giản.
- **Mở rộng hàng ngang (Scale-out/Horizontal):** Thêm các máy chủ mới vào hệ thống phân tán. Ưu điểm: Hiệu suất/giá thành tốt, sử dụng phần cứng phổ thông (commodity).

3.6.2 Kiến trúc Đa thuê bao (Multitenancy)

Có 4 cấp độ triển khai đa thuê từ thấp đến cao:

- **Cấp 1: Các thực thể tùy chỉnh (Ad hoc):** Mỗi khách hàng có phiên bản phần mềm riêng. Khó quản lý và bảo trì.
- **Cấp 2: Các thực thể có thể cấu hình:** Tất cả khách hàng dùng chung một phiên bản chương trình, tùy chỉnh qua cấu hình (Logo, workflow).
- **Cấp 3: Thực thể đa thuê hiệu quả:** Chia sẻ chung cả phiên bản chương trình và thực thể đang chạy (Running instance).
- **Cấp 4: Thực thể đa thuê mở rộng:** Kết hợp tất cả đặc tính trên và chạy trên một cụm máy tính, cho phép mở rộng gần như không giới hạn (Ví dụ: Gmail, Yahoo Mail).

3.6.3 Phát hiện lỗi (Failure Detection)

Hạ tầng đám mây cần giám sát liên tục để phát hiện các thực thể ứng dụng bị lỗi thông qua hai kỹ thuật:

- **Heartbeats (Nhịp đập):** Ứng dụng định kỳ gửi tín hiệu đến dịch vụ giám sát. Nếu thiếu một số tín hiệu liên tiếp, ứng dụng được coi là đã lỗi.
- **Probes (Thăm dò):** Dịch vụ giám sát chủ động gửi các yêu cầu nhẹ đến ứng dụng. Nếu không phản hồi, ứng dụng được coi là thất bại.

Đánh đổi: Giữa tốc độ phát hiện (nhanh) và độ chính xác (tránh nhầm lẫn do quá tải tạm thời).

3.6.4 Phục hồi ứng dụng

Sau khi phát hiện lỗi, hệ thống cần khôi phục các yêu cầu cũ:

- **Mô hình Checkpoint/Restart:**

- Định kỳ lưu trạng thái hệ thống. Khi lỗi, ứng dụng khởi động lại từ trạng thái lưu gần nhất.
- Thách thức: Phải lưu toàn bộ bộ nhớ và xử lý nhất quán với dữ liệu đang truyền trên mạng.

- **Mô hình Giao dịch (Transactional):**

- Khi một thực thể lỗi, các thực thể khác hủy bỏ công việc đang dở dang liên quan.
- Khi thực thể đó khởi động lại, nó sẽ chạy lại tất cả các giao dịch đang thực hiện.

**Thank you for
your attention!**

